

DATA MODEL · PROPOSED · NOT YET PROVISIONED

Bluedart Escalation Schema

PostgreSQL schema and end-to-end data flow for the Zoho Desk → Bluedart escalation automation — what gets written when an escalation mail goes out, what gets written when Rahul replies, and how that reply finds its way back onto the right ticket.

ENGINE	HOST	TABLES	REPLACES	STATUS
PostgreSQL 16	209.38.120.154 · localhost	8	3 JSON files	Awaiting approval

READ THIS FIRST

Why `ticket_id` is not a column on the email table

You asked for the ticket ID to sit on the main email row and act as the key into the reply table. That works when one email concerns one ticket — but this pipeline doesn't work that way, and building it as though it did would corrupt the data on day one.

One escalation email carries many tickets. The daily mapping report is a table with a row per AWB, and `thread_ticket_map()` in `main.py` already records a map of `{ticket_number → ...}` per sent mail, not a single id. **One reply carries many statuses** — Rahul answers several AWBs in one message, and `parse_status_table()` pulls a row per ticket out of it.

Both relationships are therefore many-to-many at the message level. Putting `ticket_id` directly on `emails_sent` would force one row per ticket per email, duplicating the subject, body and recipients across every line; putting it on `email_replies` would silently discard every status but the first.

What replaces it

Two junction tables carry the ticket grain: `email_tickets` (which tickets an email asked about, plus the latest answer for each) and `reply_ticket_statuses` (every answer ever received, one row per ticket per reply). **Your requirement is met in full** — the relationship is still keyed on ticket number, the latest status is still cached for fast reads, and the history is still complete. It simply lives at the grain where it is actually true.

FIGURE 1 • DATA FLOW

End-to-end flow, outbound and inbound

Outbound runs left→right along the top; inbound runs right→left along the bottom, so the write-back arrow visibly closes the loop. Rounded boxes with a coloured border are tables; dashed boxes are systems outside our control.

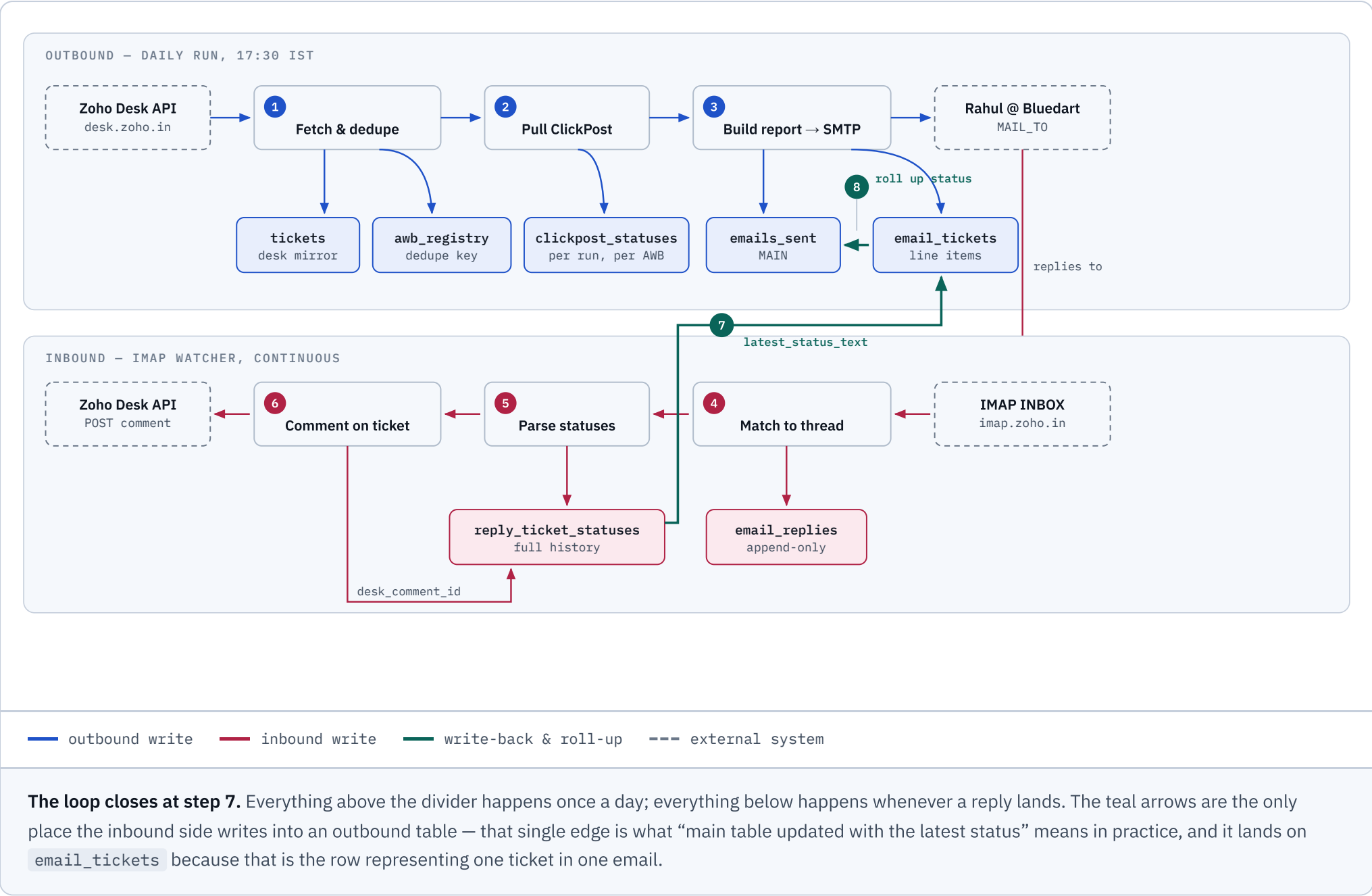
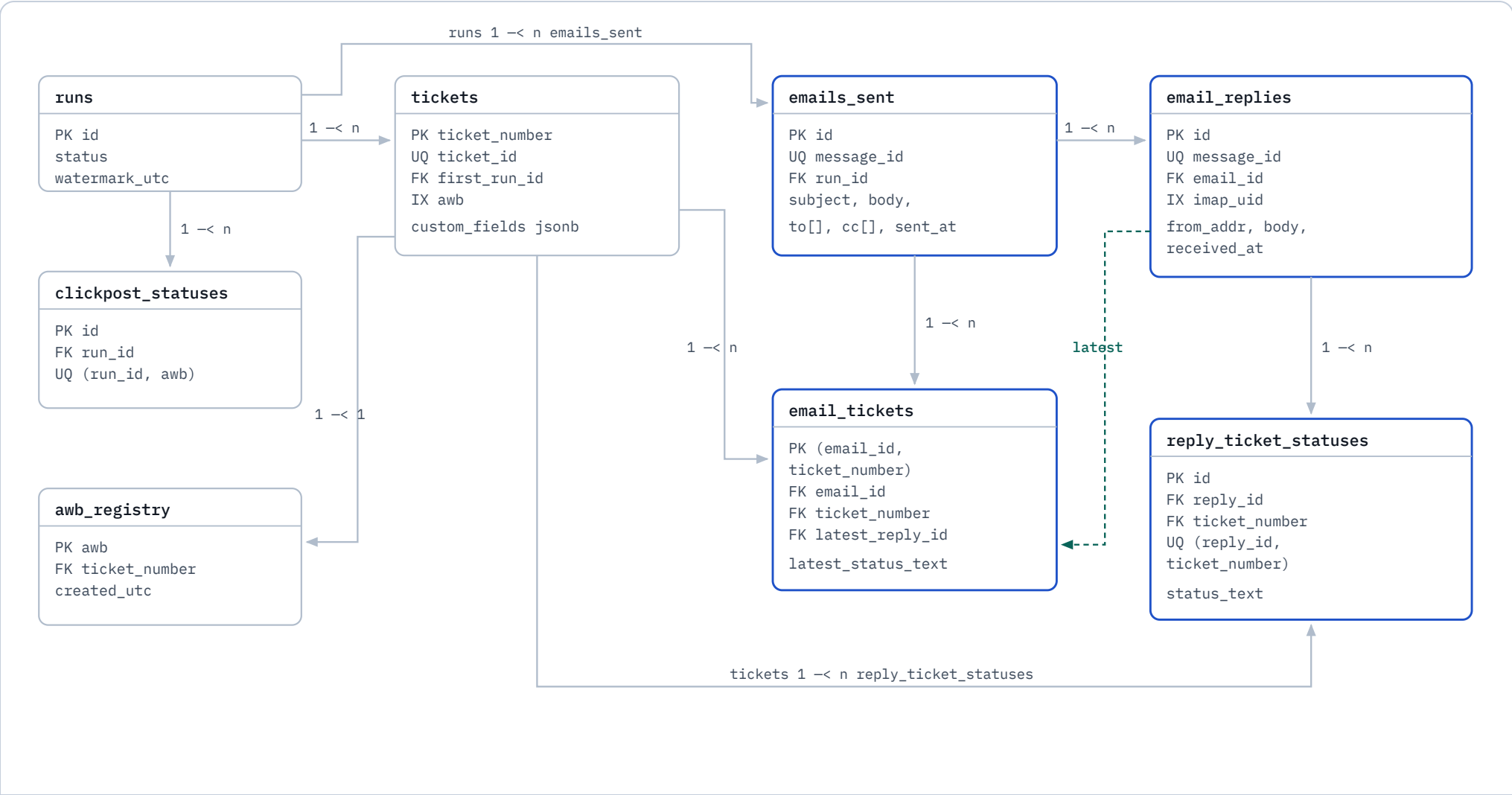


FIGURE 2 • ENTITY RELATIONSHIPS

Keys and cardinality

Primary and foreign keys only — full column lists are in the reference below. 1 —< n reads “one row here, many rows there”.



--- denormalised cache pointer — the four email / reply tables

Ticket number is the spine. `tickets.ticket_number` is the natural key — it is what the Desk UI shows, what Rahul quotes in his Status table, and what `locate_target()` already matches on. Every other table reaches a ticket through it. The one dashed edge is a cache: `email_tickets.latest_reply_id` duplicates information derivable from `reply_ticket_statuses`, bought deliberately so that the “what is still outstanding” query needs no aggregation.

STAGE TRACE

Exactly what is written, at each step

1

Fetch and dedupe tickets

The daily run opens a `pending` row, pulls Desk tickets created since the watermark plus any modified since the last sweep, filters to open Bluedart logistics tickets, and dedupes by AWB keeping the earliest ticket.

```
INSERT runs (status='pending') · UPSERT tickets × N · UPSERT awb_registry
```

2

Pull ClickPost statuses

The browser session logs in (OTP read over IMAP), runs a bulk report for this run's AWBs, downloads it and parses the export.

```
INSERT clickpost_statuses (run_id, awb, status, edd, raw) · UPDATE runs.stage_reached
```

3

Build the report and send it

Rows surviving the merge become the mapping table in the escalation mail. Once SMTP accepts the message, the mail and its line items are recorded and the run commits, advancing the watermark.

```
INSERT emails_sent (message_id, subject, from_addr, to_addrs, cc_addrs, body_html, body_text, sent_at, status='awaiting_reply') · INSERT email_tickets × N · UPDATE runs (status='committed')
```

4

A reply arrives — match it to a thread

The watcher wakes, fetches UIDs above the stored high-water mark, and matches each on `In-Reply-To` / `References` against `emails_sent.message_id`, falling back to the exact dated subject.

```
INSERT email_replies (email_id, message_id, in_reply_to, imap_uid, from_addr, subject, body_html, body_text, received_at, fetched_at, match_method)
```

5

Parse the Status table

The reply's HTML table becomes one status row per ticket. Every row is stored, including for tickets already answered by an earlier reply — this table is append-only and nothing in it is ever overwritten.

```
INSERT reply_ticket_statuses (reply_id, ticket_number, awb, status_text, matched_via) × N
```

6

Comment on the Zoho Desk ticket

A private comment goes onto each ticket through the Desk API. The returned comment id is written back as proof of delivery; a failure is recorded rather than swallowed, so it stays retryable.

```
UPDATE reply_ticket_statuses SET desk_comment_id, desk_posted_at – or post_error
```

7

Update the latest status

The newest status for each ticket is cached onto its line item. This is the “main table updated with the latest response” step — and because step 5 already stored the row permanently, overwriting the cache loses nothing.

```
UPDATE email_tickets SET latest_status_text, latest_reply_id, answered_at, line_status='answered'
```

8

Roll the email status up

When every line item on an email is answered, the email is complete. When only some are, it stays open so the 15:00 IST follow-up keeps chasing the remainder.

```
UPDATE emails_sent SET status = 'completed' | 'partially_answered', completed_at
```


REFERENCE

Tables and columns

Types are PostgreSQL. **PK** primary key **FK** foreign key **UQ** unique **IX** indexed.

emails_sent MAIN TABLE One row per outbound email — escalation or follow-up			
COLUMN	TYPE	KEY	NOTES
id	bigserial	PK	Surrogate key. Foreign keys point here, not at the message id.
message_id	text	UQ	RFC 5322 Message-ID. The join key for replies — matched against the reply's <code>In-Reply-To</code> .
run_id	bigint	FK	→ <code>runs.id</code> . Which pipeline run produced this mail.
kind	text		<code>escalation</code> <code>followup</code> , constrained by CHECK.
subject	text		The exact dated subject — also the fallback reply match.
from_addr	text		Sender.
to_addrs	text[]		Recipients. An array rather than CSV — queryable, and cannot mis-split on a comma inside a display name.
cc_addrs	text[]		Copy list; may be empty.
body_html	text		The message as sent, mapping table included.
body_text	text		Plain-text alternative.
ticket_count	int		Denormalised count of line items, for cheap listings.
sent_at	timestampz	IX	When SMTP accepted it.
status	text	IX	<code>awaiting_reply</code> <code>partially_answered</code> <code>completed</code> , rolled up from the line items.
followup_sent_at	timestampz		Null until the 15:00 IST nudge fires. One per email.
completed_at	timestampz		Set when the last line item is answered.

email_tickets			
LINE ITEMS · LATEST STATUS			
One row per ticket per email			
COLUMN	TYPE	KEY	NOTES
email_id	bigint	PK FK	→ emails_sent.id , ON DELETE CASCADE .
ticket_number	text	PK FK	→ tickets.ticket_number . The composite primary key is what makes double-commenting impossible, replacing the in-memory if key in book guard.
awb	text		AWB as reported in this mail — a snapshot, not a live join.
line_status	text	IX	awaiting_reply answered . Drives the follow-up query.
latest_status_text	text		Rahul's most recent remark, verbatim. Overwritten by each new reply — the history lives in reply_ticket_statuses .
latest_reply_id	bigint	FK	→ email_replies.id . Which reply produced the cached status.
answered_at	timestampz		First time this line received any answer.
desk_comment_id	text		Proof the comment reached Zoho Desk.
desk_posted_at	timestampz		When it was posted.

email_replies REPLY TABLE · APPEND-ONLY One row per inbound reply message			
COLUMN	TYPE	KEY	NOTES
id	bigserial	PK	Surrogate key.
email_id	bigint	FK IX	→ emails_sent.id . This is the relationship you asked for , resolved one level up — at the message the reply answers, rather than at a single ticket.
message_id	text	UQ	Idempotency key. Re-reading the same reply cannot double-insert; replaces the seen_replies[] array.
in_reply_to	text		Raw header, kept so a mismatch can be audited.
imap_uid	bigint	IX	The highest UID seen becomes the next SEARCH UID n:* floor — this is what removes the per-cycle inbox rescan.
from_addr	text		Who replied.
to_addrs	text[]		Recipients on the reply.
subject	text		As received.
body_html	text		The full reply, kept verbatim.
body_text	text		Plain-text rendering.
received_at	timestampz		The Date header — when Rahul sent it.
fetches_at	timestampz		When the watcher pulled it. The gap between these two is your reply latency , measurable at last.
match_method	text		in_reply_to references subject_and_table .
status_rows_found	int		Zero means a reply with no parseable Status table — worth alerting on.

reply_ticket_statuses FULL HISTORY · APPEND-ONLY One row per ticket per reply — never overwritten			
COLUMN	TYPE	KEY	NOTES
id	bigserial	PK	Surrogate key.
reply_id	bigint	FK	→ email_replies.id , ON DELETE CASCADE .
ticket_number	text	FK IX	→ tickets.ticket_number . Indexed with created_at DESC , so “history for this ticket” is a single index scan.
awb	text		AWB quoted in the reply row.
status_text	text		The remark, verbatim. Every version is kept — a later reply adds a row, it never replaces one.
matched_via	text		How the ticket was resolved: thread registry awb_fallback .
desk_comment_id	text		Null until posted to Desk.
desk_posted_at	timestamptz		When the comment landed.
post_error	text		Recorded instead of swallowed, so a failed post can be retried.
created_at	timestamptz		When this row was parsed.
—	constraint	UQ	(reply_id, ticket_number) — one status per ticket per reply.

tickets SUPPORTING Mirror of the Desk ticket — the spine every other table joins to			
COLUMN	TYPE	KEY	NOTES
ticket_number	text	PK	Natural key — what Desk shows and what Rahul quotes.
ticket_id	text	UQ	Internal Desk id, required by the comment API. Currently re-derived by paging up to 2,000 tickets; stored once here instead.
created_utc	timestamptz	IX	Drives the created watermark.
modified_utc	timestamptz	IX	Drives the late-edit sweep.
subject, status, status_type, owner, department, priority	text		Core Desk fields. <code>status_type</code> is tested rather than the label, so a renamed status cannot leak closed tickets back in.
awb	text	IX	Normalised AWB.
courier, logistics_class, vinculum_edd, states, pending_with_dept	text / date		The custom fields, read by display label since the <code>cf_*</code> API names do not match them.
custom_fields	jsonb		The raw block. A new Desk field needs no migration.
first_seen_at, last_seen_at	timestamptz		Upsert bookkeeping.
first_run_id	bigint	FK	→ <code>runs.id</code> . Which run first saw it.

runs SUPPORTING Replaces <code>state.json</code> — run ledger and watermark			
COLUMN	TYPE	KEY	NOTES
id	bigserial	PK	Surrogate key.
started_at, finished_at	timestamptz		Run span.
status	text		<code>pending</code> <code>committed</code> <code>failed</code> . A partial unique index enforces at most one pending run — something a JSON file cannot express.
stage_reached	text		<code>desk</code> <code>clickpost</code> <code>merge</code> <code>mapping</code> <code>mailed</code> .
attempts	int		Retry counter for the pending block.
watermark_ticket, watermark_utc	text / timestamptz		The created watermark. Its current value is the newest <code>committed</code> row.
modified_utc	timestamptz		Late-edit sweep watermark.
rows_count, mailer_sent	int / bool		Outcome of the run.

awb_registry SUPPORTING Replaces <code>awb_registry.json</code> — one keeper ticket per AWB			
COLUMN	TYPE	KEY	NOTES
<code>awb</code>	text	PK	Normalised. Stops the same shipment being escalated twice.
<code>ticket_number</code>	text	FK	→ <code>tickets</code> . The earliest-created ticket wins; a backfill turning up an earlier one replaces it.
<code>created_utc</code>	timestampz		Kept denormalised so the keeper comparison needs no join.
<code>registered_at</code>	timestampz		When it entered the registry.

clickpost_statuses SUPPORTING Courier data per run — currently deleted after every run			
COLUMN	TYPE	KEY	NOTES
<code>id</code>	bigserial	PK	Surrogate key.
<code>run_id</code>	bigint	FK	→ <code>runs.id</code> .
<code>awb</code>	text	IX	Shipment.
<code>status, edd</code>	text / date		Courier status and expected delivery date at the time of the run.
<code>raw</code>	jsonb		Full export row, so a parsing change can be replayed against history.
<code>fetches_at</code>	timestampz		Snapshot time.
<code>-</code>	constraint	UQ	<code>(run_id, awb)</code> .

DESIGN RATIONALE

What the constraints buy you

Several correctness guards currently live in Python state rebuilt from a JSON file on every run. Each becomes a database constraint instead, which means it holds even when two processes run at once — as they will, once the incoming-mail webhook lands.

- **No duplicate comment on a ticket.** Today: `if key in book` against an in-memory dict. Now: the composite primary key on `email_tickets`.
- **No reprocessing the same reply.** Today: a `seen_replies[]` array scanned linearly. Now: `UNIQUE (message_id)` on `email_replies`.
- **No two runs in flight.** Today: nothing — a second run overwrites the pending block. Now: a partial unique index on `runs(status) WHERE status='pending'`.
- **History survives an overwrite.** Today: a later reply replaces the earlier remark in memory. Now: append to `reply_ticket_statuses`, cache the latest on `email_tickets`.
- **Late replies stop vanishing.** Today: once a thread flips to `completed` it is skipped entirely, so a follow-up remark is silently dropped. Now: the reply still matches on `message_id` and appends a row regardless of status.

The reply-latency fix comes almost free

Persisting `email_replies.imap_uid` lets the watcher search `UID <last+1>:*` instead of re-walking `SINCE <date>` for every open thread on every cycle. That shrinks the processing gap behind the occasional 15-minute delay — not the purpose of this schema, but a direct consequence of it.

Retention: keep everything — decided

No purge job, no TTL, nothing expires. `tickets`, `emails_sent.body_*` and `email_replies.body_*` retain customer data indefinitely, where today it is deleted after every run. Two consequences worth stating once, since the decision is made:

- **Growth is dominated by message bodies**, not by row count. PostgreSQL compresses any text value over roughly 2 KB out of line automatically, so an HTML mail body typically lands on disk at a fraction of its size — growth should stay modest without any intervention.
- **Backups now carry PII permanently**. The nightly `pg_dump` should be written to a directory the automation user alone can read, and never to a location that syncs off the box.

One blocker remains

SSH access. Nothing has been provisioned. PostgreSQL is not installed and no table exists on 209.38.120.154.

Proposed data model · Zoho Desk → Bluedart escalation automation · not yet provisioned